DEVELOPMENT STORY

# AI 에이전트 팀으로 차세대 추천 시스템을 만들기까지

소비자 *GPU 1*대, *3*인 팀, 그리고 *AI* 협업의 기록

*"AI*가 코드를 쓰지만,
설계 판단은 사람이 한다*."*

인프라 예산 없이, 소비자 GPU 1대와 AI 에이전트들의 조합으로
18-task, 7-expert PLE+adaTT 추천 시스템을 구축한 과정을 기록한다.

AIOps PLE Platform

# 프로젝트 배경

## 팀 구성과 제약조건

프로젝트 팀은 3명이었다. 데이터사이언티스트 겸 PM/리드 아키텍트 1명과 엔지니어 2명. 전용 인프라 예산은 없었고, 개발에 사용할 수 있는 GPU는 로컬 PC에 장착된 소비자용 RTX 4070 (12GB VRAM) 1대가 전부였다.

> 제약조건 요약
>
> - 팀 규모: 3명 (PM/리드 아키텍트/데이터사이언티스트 1 + 엔지니어 2)
> - 인프라: 전용 GPU 서버 없음. RTX 4070 12GB 1대 (로컬)
> - 예산: 인프라 구매 예산 없음. AWS SageMaker spot 인스턴스 활용
> - 일정: 기존 ALS 기반 추천 시스템을 대체할 차세대 시스템 구축

## 인프라 제약의 현실

조직으로부터 받은 지원은 사실상 전무했다. GPU가 필요하다고 요청해도 "어쩔 수 없다"는 답변뿐이었고, AI 도구 구독료(Claude Code, Gemini, Cursor), 부속기기, 식비 등 프로젝트에 소요된 모든 비용은 PM/리드 아키텍트 개인 자금으로 충당했다.

데이터 수집 환경도 열악했다. Spark나 Impala로의 전환 요청이 거부되어 HIVE 기반의 병목 환경에서 작업해야 했고, 이를 극복하기 위해 병렬 쿼리 로직을 직접 설계하여 네트워크 대역폭을 최대한 활용했다.

작업 공간은 열순환이 되지 않는 서버실 옆 공간으로, 적절한 냉방도 제공되지 않았다.

함께 일한 팀원 2명은 정식 계약이 아닌 청년 자문위원 자격의 서포터즈로, 대학 졸업 후 취업을 준비하는 과정에서 프로젝트에 참여했다.

이 모든 제약이 오히려 설계 철학을 강화했다: "가용 자원이 극히 제한적일 때, 아키텍처 효율성과 도구 선택이 결정적으로 중요해진다."

## 온프렘 시스템 규모

온프렘 시스템은 단순한 프로토타입이 아니라 프로덕션 규모의 시스템이었다. 80개 이상의 Airflow DAG, Champion-Challenger 모델 경쟁, 주간 자동 재학습, 734D 피처 텐서, 16개 동시 태스크, 62개 데이터 테이블 인제스천을 포함한다. 이 규모의 시스템을 3명이 구축한 것 자체가 AI 에이전트 활용의 결과물이다.

## 프로젝트 목표

기존 금융 상품 추천 시스템은 ALS (Alternating Least Squares) 기반의 협업 필터링이었다. 이를 PLE (Progressive Layered Extraction) + adaTT (Adaptive Task Transfer) 아키텍처 기반의 멀티태스크 딥러닝 추천 시스템으로 교체하는 것이 목표였다. 18개 태스크를 7개 전문가 네트워크가 처리하며, 태스크 간 관계를 명시적으로 모델링하는 구조다.

## 아키텍처 의사결정 여정

최종 아키텍처에 도달하기까지 여러 후보를 탐색하고 기각하는 과정을 거쳤다.

기존 시스템은 ALS 기반 협업 필터링이었다. 첫 번째 대안으로 **Black-Litterman** 모델을 검토했다. 포트폴리오 이론에서 온 이 모델은 전문가 의견(뷰)과 시장 균형을 베이지안 업데이트로 결합한다. 그러나 사후분포로 혼합된 결과에서 "어떤 모델이 얼마나 기여했는가"를 분리할 수 없었다. 금융 현장에서는 고객, 영업점, 규제기관 모두에게 추천 근거를 설명해야 하는데, 베이지안 업데이트가 개별 모델의 기여를 불투명하게 만들어 비즈니스 설명이 불가능했다.

두 번째 대안은 모델 앙상블이었다. N개 모델을 독립적으로 학습시키고 결과를 결합하는 방식이다. 그러나 N개의 관리 포인트, N배의 서빙 비용이 발생하고, "MLP 3번이 28% 기여했다"는 식의 설명은 비즈니스적으로 무의미했다. 비용과 설명 가능성 모두에서 문제가 있었다.

이 과정에서 핵심적인 리프레이밍이 발생했다: 전문가들을 모델 밖에서 결합하는 것이 아니라, 단일 모델 안에서 결합하면 어떨까? 이것이 PLE의 이종 전문가 아키텍처로의 전환 계기였다.

태스크 그룹 설계에서도 시행착오가 있었다. 초기에는 **GMM** 클러스터 서브헤드 방식을 시도했다 — K개 클러스터 × T개 태스크 = $K \times T$ 복잡도 폭발이 발생했다. 이를 4개의 Financial DNA 태스크 그룹(상품 보유 확률, 다음 상품, 고객 가치, 이탈 위험)으로 전환하여 구조를 안정화했다.

---

# AI 에이전트 조직화

## Phase별 AI 도구 운용

프로젝트의 각 단계에서 서로 다른 AI 도구를 전략적으로 배치했다. 각 도구의 강점을 살려 역할을 분담하는 것이 핵심이었다.

**Phase A:** 아이디에이션 **(Gemini)**

초기 컨셉 탐색과 브레인스토밍에는 Gemini를 활용했다. 광범위한 지식 베이스를 바탕으로 아키텍처 후보군을 빠르게 스캔하고, 다양한 접근법의 장단점을 비교하는 데 효과적이었다. ALS 대체 옵션, Black-Litterman 탐색, 모델 앙상블 접근법 등 다양한 아키텍처 후보를 Gemini와의 대화를 통해 탐색했다.

가장 큰 가치를 발휘한 것은 학제간 피처 아이디어의 탐색이었다. “화학 반응속도론으로 소비 행동을 설명할 수 있는가?”, “상품 채택이 전염병 확산과 구조적으로 동등한가?” 같은 질문을 던지며, 금융 고객 행동과 구조적으로 동형인 문제를 가진 학문 분야를 체계적으로 식별했다. PM/리드 아키텍트가 도메인 전문성(FRM, 신용 분석 경력)을 제공하고, Gemini가 학제간 연결고리를 제안하는 방식으로 협업이 이루어졌다. 이 과정에서 “구조적 동형사상”이라는 핵심 개념이 자연스럽게 부상했다.

Gemini의 광범위한 지식은 특정 기술의 깊이보다는 “어떤 분야에서 비슷한 문제를 이미 풀었는가”를 빠르게 탐색하는 데 최적이었다. 11개 학문 분야에서 피처를 도입하겠다는 설계 방향이 이 단계에서 확립되었고, 이후 모든 기술적 결정의 근간이 되었다.

**Phase B:** 기술 검증 **(Claude Opus)**

아이디에이션 결과를 구체적인 아키텍처로 발전시키는 단계에서는 Claude Opus를 사용했다. 수학적 엄밀성이 필요한 loss function 설계, 데이터 리키지 검증, 정규화 파이프라인 설계 등 기술적 깊이가 요구되는 작업에 집중했다.

각 전문가의 실현 가능성을 하나씩 검증했다: “HGCN이 MCC 계층 구조에서 작동하는가?”, “Mamba가 17개월 시퀀스에 충분히 효율적인가?” 같은 질문에 대해 Opus와 심층적인 기술 토론을 진행했다. PLE vs MMoE의 트레이드오프 분석, adaTT의 loss-level vs representation-level 전이 비교 등 아키텍처 수준의 깊은 분석이 이 단계에서 이루어졌다. 특히 동질적 MoE에서 발생하는 전문가 붕괴(expert collapse) 문제를 Opus와의 대화에서 먼저 식별한 후, NeurIPS 2024의 sigmoid gate 논문을 발견하는 계기가 되었다.

Opus는 가정을 도전하는 데도 효과적이었다. “Black-Litterman이 정말 적합한가?”라는 반론을 제기하여 PLE로의 전환을 촉진했다. 이 단계에서 19개의 기술 참조 문서(.typ 파일)가 Opus와의 공동 작업으로 작성되었으며, 이 문서들은 이후 구현 단계에서 각 에이전트가 참조하는 설계 명세서 역할을 했다.

**Phase C:** 코드 환경 정비 **(Cursor)**

GitHub 기반의 코드 환경 구성, 프로젝트 구조 설계, 초기 보일러플레이트 생성은 Cursor로 수행했다. IDE 통합 환경에서의 빠른 코드 네비게이션과 리팩토링이 강점이었다.

이 단계에서 6개의 초기 설계 문서(00-09 아키텍처 명세)가 작성되었고, 가장 중요한 산출물은 CLAUDE.md 가드레일의 수립이었다. Config-driven 원칙, 관심사 분리, 리키지 방지 규칙 등 이후 모든 AI 에이전트가 따라야 할 “헌법”이 코드 한 줄 작성되기 전에 확립되었다. 이것은 의도적인 순서였다 — 가드레일이 먼저이고, 에이전트 실행이 그 다음이다.

**Phase D:** 병렬 구현 **(Claude Code — Opus/Sonnet)**

본격적인 구현 단계에서는 각 팀원이 AI 에이전트의 “팀장” 역할을 맡았다. Claude Code 환경에서 Opus와 Sonnet을 병렬로 운용하여 서로 다른 모듈을 동시에 구현했다. 이 단계가 가장 집약적이었으며, 3명의 인간이 각각 AI 에이전트 팀을 이끌었다.

PM/리드 아키텍트의 AI 팀은 아키텍처 수준의 결정(PLE config, adaTT 태스크 그룹, 로짓 전이 설계)에는 Opus를, 빠른 코드 구현(generator, adapter, pipeline runner)에는 Sonnet을 배치했다. 이 과정에서 label leakage 3건 탐지, FP16 NaN 원인 4가지 진단, GPU 활용률 최적화 등 핵심적인 디버깅 세션이 이루어졌다. 엔지니어 1의 AI 팀은 데이터 수집 파이프라인, HIVE 병렬 쿼리 로직, 10개 generator(TDA,

HMM, Mamba, Graph, GMM 등)의 피처 엔지니어링, 피처-비즈니스 역매핑 레지스트리를 담당했다. 엔지니어 2의 AI 팀은 모델 학습, 수학적 검증, 지식 증류(PLE에서 LGBM으로)를 담당했다.

각 팀이 병렬로 작업하면서도 일관성을 유지할 수 있었던 것은 Phase C에서 수립한 CLAUDE.md 가드레일과 인터페이스 계약 검증 프로세스 덕분이었다. 파일 A가 저장하는 키와 파일 B가 읽는 키가 일치하는지를 매 병렬 작업 후 반드시 검증했고, 이를 통해 통합 시 발생할 수 있는 인터페이스 불일치를 사전에 방지했다.

**Phase E:** 실험 + 논문 **(Claude Code Extension)**

ablation 실험 단계에서는 Claude Code를 실시간 모니터링 도구로 활용했다. ablation 진행 상황, GPU 활용률, 에러 탐지를 실시간으로 수행했다. 이 과정에서 PLE toggle 버그를 라이브 디버깅으로 발견했다 — `use_ple=false` 설정이 전문가 구성 자체를 변경하여 공정한 비교가 불가능했던 문제였다.

실험 대기 시간을 활용한 문헌 조사도 이루어졌다. PLE의 val_loss가 수렴하지 않는 현상을 관찰하던 중, Opus와의 대화를 통해 softmax gate의 경쟁적 특성이 이종 전문가 간 수렴을 방해한다는 가설을 세웠고, NeurIPS 2024의 sigmoid gate 논문을 찾아 이론적 근거를 확보했다. 실험 결과 분석과 논문 작성이 동시에 진행되는 방식이었다.

논문 작성 단계에서는 4편의 논문(영문/한국어), 22개의 기술 문서가 Claude와의 반복적 작업으로 생성되고 다듬어졌다. 개발 스토리 자체도 Claude와 함께 프로젝트 과정을 되돌아보며 작성되었다. 이 단계에서 AI는 단순한 텍스트 생성기가 아니라, 프로젝트의 의미를 함께 구성하는 사고 파트너 역할을 했다.

## 문서 생산 규모

AI 에이전트와의 협업은 코드 구현에 그치지 않았다. 온프렘 프로젝트에서만 260개 이상의 문서(설계서 28개, 기술참조서 19개, 코드 리뷰 16건, 보고서 95건, 가이드 5개)가 생산되었으며, 총 30MB 이상의 기술 문서가 작성되었다. 이 중 상당수는 AI와 공동 작성 또는 AI가 초안을 작성하고 인간이 검수하는 방식으로 제작되었다. 특히 “소네트 작업 검증 리포트”는 Opus가 Sonnet의 코드를 검증하는 AI 간 리뷰 프로세스를 보여주며, “Claude Code Opus용 500+ 항목 체크리스트”는 AI 에이전트에게 체계적 검수 업무를 위임하는 방법론을 보여준다.

## 메모리 뱅크와 가드레일 시스템

온프렘 프로젝트에서 확립된 AI 관리 체계는 AWS 프로젝트에도 그대로 이식되었다. memory-bank 시스템(8개 컨텍스트 파일: projectbrief, activeContext, progress, techContext, productContext, systemPatterns, tasks, style-guide)으로 세션 간 맥락을 유지하고, .claude/RULES.md로 코딩 규칙을 강제하며, .cursorrules와 동기화하여 Cursor AI와 Claude Code가 동일한 가드레일을 따르도록 했다. 심지어 Claude, Codex, Vertex AI 세 플랫폼에서 자동화 실험 브랜치(exp/claude-auto-*, exp/codex-auto-*, exp/vertex-auto-*)를 운영하여 AI 도구 간 비교 실험도 수행했다.

## AI 협업에서 발견된 핵심 패턴

프로젝트 전 과정에서 AI 협업의 반복적 패턴이 드러났다. 이 패턴들은 의도적으로 설계된 것이 아니라, 실제 작업 과정에서 자연스럽게 발현된 것이다.

**1. "AI는 HOW, 인간은 WHAT과 WHY를 결정한다"**

: AI가 코드와 텍스트를 생성하지만, 아키텍처 결정은 인간이 내린다. 구조적 동형사상이라는 핵심 통찰은 인간-AI 대화에서 발현되었지만, 그것을 설계 원칙으로 채택한 것은 인간의 판단이었다.

**2. "에이전트 전에 가드레일을 세운다"**

: CLAUDE.md는 코드가 아니라 코드 이전에 작성되었다. 헌법이 먼저이고 입법이 나중인 것처럼, 가드레일이 먼저이고 에이전트 실행이 나중이다.

**3. "이종 AI = 이종 전문가"**

: 모델의 이종 전문가 설계가 개발 도구 선택에도 그대로 적용되었다. 각 AI 도구가 특화된 역할을 수행하면서, 단일 도구로는 달성할 수 없는 품질과 속도를 확보했다.

**4. "메모리 뱅크로 연속성 확보"**

: 세션 간 맥락 보존을 위한 영속 파일 시스템이 AI 에이전트의 가장 큰 약점인 "맥락 망실"을 극복하는 핵심 메커니즘이었다.

**5. "AI로 빠르게 실패한다"**

: leakage, FP16 NaN, ablation 필터 미작동 등 수동으로는 수일이 걸릴 버그를 AI 에이전트와 함께 수분 내에 탐지하고 수정했다. 빠른 실패가 빠른 학습으로 이어졌다.

**6. "AI는 코드 머신이 아니라 사고 파트너"**

: 구조적 동형사상이라는 통찰, sigmoid gate 가설, 전문가 붕괴 문제의 식별 등 프로젝트의 핵심적인 지적 돌파구가 인간-AI 대화에서 나왔다.

| Phase | AI 도구 | 역할 |
|---|---|---|
| A. 아이디에이션 | Gemini | 컨셉 탐색, 아키텍처 후보 스캔, 학제간 브레인스토밍 |
| B. 기술 검증 | Claude Opus | 수학적 검증, loss 설계, 리키지 분석, 기술 문서 19편 공동 작성 |
| C. 환경 정비 | Cursor | GitHub 구조, CLAUDE.md 가드레일, 설계 문서 6편 |
| D. 병렬 구현 | Claude Code (Opus/Sonnet) | 3인 x AI 팀 병렬 코딩, 디버깅, 10개 generator 구현 |
| E. 실험 + 논문 | Claude Code Extension | 실시간 모니터링, 문헌 조사, 논문 4편 + 기술 문서 22편 |

# 품질 관리 전략

## CLAUDE.md 가드레일

AI 에이전트가 생성하는 코드의 품질을 보장하기 위해, 프로젝트 루트에 CLAUDE.md 파일을 두어 가드레일을 설정했다. 이 파일은 AI 에이전트가 매 세션 시작 시 읽어들이는 시스템 지침이다.

> **CLAUDE.md 핵심 가드레일**
>
> - **Config-Driven** 원칙: 모든 파라미터는 YAML에서 읽는다. 하드코딩 금지.
> - 관심사 분리: Adapter는 데이터 변환만, Runner는 파이프라인만, train.py는 학습만.
> - 데이터 리키지 방지: Scaler는 train split에서만 fit. temporal split에 gap_days 필수.
> - 코드 검수 **4**단계: 컴파일 검증 → 인터페이스 계약 검증 → 하드코딩 스캔 → 관심사 분리 검증.

## 메모리 뱅크

AI 에이전트는 대화 세션이 끝나면 맥락을 잃는다. 이 문제를 해결하기 위해 "메모리 뱅크" 시스템을 도입했다. 세션 진행 상황, 설계 결정 사유, 피드백 이력을 구조화된 마크다운 파일로 관리하여, 새 세션이 시작될 때 AI가 이전 맥락을 빠르게 복원할 수 있게 했다.

## 인터페이스 계약 검증

병렬 에이전트가 서로 다른 모듈을 동시에 수정할 때 가장 큰 위험은 인터페이스 불일치다. 파일 A가 저장하는 키 이름과 파일 B가 읽는 키 이름이 달라지는 문제를 방지하기 위해, 병렬 작업 후에는 반드시 "인터페이스 계약 검증" 단계를 수행했다. cross-file 키/필드 매핑 테이블을 자동 생성하여 불일치를 사전에 탐지했다.

---

# 기술적 도전과 해결

## Label Leakage 3건 발견 및 수정

모델 학습 초기에 비정상적으로 높은 성능이 관측되었다. 원인 분석 결과, 3건의 label leakage를 발견했다.

> **Leak 1 —** 중복 컬럼: `has_nba_1` 컬럼이 레이블과 완전 중복(상관계수 1.0)으로 존재했다. 레이블 재파생 전에 EXCLUDE 처리가 필요했다.
> **Leak 2 —** 파일 로딩 순서: `ground_truth.parquet`가 `benchmark.parquet` 대신 로딩되었다. glob의 알파벳 정렬로 인해 정답 파일이 먼저 선택된 것이다. 해당 파일을 하위 디렉토리로 이동하여 해결했다.
> **Leak 3 — Generator** 입력 오염: GMM, model_derived 등의 generator가 레이블 컬럼을 입력으로 사용하여 AUC=1.0이 관측되었다. `label_cols`를 자동 제외하는 로직을 추가했다.

LeakageValidator를 학습 전 필수 단계로 추가하고, CLAUDE.md 가드레일에 검증 규칙을 명시하여 재발을 방지했다.

## Phase 2 NaN 문제 (FP16 Underflow)

Mixed precision (AMP) 학습 중 Phase 2에서 NaN loss가 발생했다. 원인은 BFloat16 환경에서의 underflow였다. 작은 gradient 값들이 FP16 범위 밖으로 떨어지면서 NaN으로 전파되었다.

> 해결 방법
>
> - BFloat16에서 `.float()` 변환 후 `.numpy()` 호출하는 패턴 적용
> - subprocess pipe deadlock 문제 동시 수정
> - GradScaler 설정 최적화로 underflow 빈도 감소

## GPU 활용률 최적화 (37% → 98%)

초기 학습 시 GPU 활용률이 37%에 불과했다. 병목은 데이터 로딩이었다.

> 최적화 단계
>
> - **batch_size** 증가: 512 → 4096 (941K 데이터에 최적화)
> - **DataLoader** 최적화: num_workers, pin_memory, prefetch_factor 조정
> - 데이터 전처리 분리: Phase 0에서 텐서를 미리 저장, 학습 시 로드만 수행
> - 결과: GPU 활용률 37% → 98%, 학습 시간 약 3배 단축

## pandas에서 DuckDB/cuDF로 전환

941K 행의 데이터를 pandas로 처리하면 메모리 사용량이 급격히 증가했다. 대규모 데이터 처리 백엔드를 DuckDB (CPU columnar)와 cuDF (GPU)로 전환하여 메모리 효율성과 처리 속도를 동시에 개선했다.

## Docker GPU Passthrough 불안정

Windows 환경에서 Docker를 통한 GPU passthrough가 불안정하게 동작했다. CUDA 버전 불일치, 드라이버 호환성 문제가 반복적으로 발생했다. 결국 Docker를 포기하고 로컬 Python 환경에서 직접 개발하는 방식으로 전환했다.

## torch CPU/CUDA 버전 충돌

conda 환경에서 torch의 CPU 빌드와 CUDA 빌드가 충돌하는 문제가 발생했다. 패키지 의존성 꼬임으로 인해 CUDA가 인식되지 않는 상황이 반복되었다. conda 캐시를 완전히 정리하고 CUDA 버전을 명시적으로 지정하여 환경을 재구성했다.

## Ground Truth 파일 오류 로드

Phase 0 실행 결과가 이상했다. 기대한 300개 이상의 피처 대신 142개만 생성되었고, 기본 피처들이 전부 누락되어 있었다. 원인 추적 결과, `benchmark_ground_truth.parquet`가 glob의 알파벳 순서로 `benchmark_v2.parquet`보다 앞에 위치하여 먼저 로드된 것이었다. Ground truth 파일에는 persona, latent 변수 등 정답 변수만 존재하여, 이를 원본 데이터로 인식한 Phase 0가 엉뚱한 피처를 생성했다. 해결 방법은 단순했다. ground truth 파일을 `ground_truth/` 하위 디렉토리로 분리하여 glob 대상에서 제외시켰다.

## Ablation 필터 미작동

24개 ablation 시나리오를 실행했는데, 모든 시나리오에서 AUC가 0.913으로 동일했다. 피처 제거가 전혀 작동하지 않은 것이다. 원인은 `feature_group_ranges`가 개별 컬럼 단위(예: `tda_global_h0_num_features`)로 저장되어 있었는데, ablation 필터는 그룹명(예: `tda_global`)으로 매칭을 시도했기 때문이다. 키 이름이 한 번도 일치하지 않아 모든 시나리오에서 피처가 하나도 제거되지 않았다. adapter에서 `feature_group_ranges`를 컬럼 단위와 그룹 레벨 양쪽으로 저장하도록 수정하여 해결했다.

## apply_ablation Schema 미갱신

Ablation 필터가 작동하기 시작한 후에도, 여전히 모든 시나리오의 성능이 동일했다. 피처를 텐서에서 제거하는 데는 성공했지만, `feature_schema["columns"]`가 원래의 316개 컬럼을 유지하고 있었다. 모델은 schema에 기록된 316차원으로 빌드되어, 제거된 피처 자리가 0으로 패딩되었다. 결국 모델 입장에서는 매번 동일한 차원의 입력을 받은 셈이었다. `apply_ablation` 함수에서 텐서 제거와 동시에 `columns`, `num_features`, `feature_group_ranges`를 함께 갱신하도록 수정하여 비로소 ablation이 정상 작동했다.

## Subprocess Pipe Deadlock

ablation 오케스트레이터가 각 시나리오를 `subprocess.run(capture_output=True)`로 실행했는데, full 시나리오에서 프로세스가 무한 대기 상태에 빠졌다. 원인은 학습 과정에서 출력하는 대량의 stdout이 파이프 버퍼(64KB)를 초과한 것이었다. 버퍼가 가득 차면 자식 프로세스가 쓰기에서 블록되고, 부모 프로세스는 자식이 끝나기를 기다리면서 교착 상태가 발생했다. stdout과 stderr를 파일로 리다이렉트하는 방식으로 전환하여 해결했다.

## BFloat16 NumPy 변환 실패

AMP 학습이 생성한 BFloat16 텐서를 validation 단계에서 `.numpy()`로 변환하려 했으나, NumPy가 BFloat16 dtype을 지원하지 않아 에러가 발생했다. 이로 인해 모든 validation metrics 계산이 실패했다. 학습 자체는 정상적으로 진행되었지만 성능을 확인할 수 없는 상태였다. 모든 텐서에 `.float()` 캐스트를 적용한 후 `.numpy()`를 호출하는 패턴으로 통일하여 해결했다.

## GradScaler Assertion 에러

Phase 2에서 모든 배치가 NaN을 생성하는 극단적 상황이 발생했다. 모든 배치가 NaN이면 backward pass가 한 번도 호출되지 않는데, `scaler.step()`은 최소 한 번의 inf check가 수행되었음을 가정하여 "No inf checks were recorded" assertion을 발생시켰다. `_scaler_backward_count`를 추적하여 0인 경우 `scaler.step()`을 건너뛰는 방어 로직을 추가했다. 이는 AMP와 multi-task loss의 조합에서 발생하는 드문 엣지 케이스였다.

## Docker 좀비 컨테이너

학습 속도가 갑자기 이전의 1/3로 떨어졌다. `nvidia-smi`로 확인한 결과, 이전 실험에서 종료되지 않은 Docker 컨테이너가 GPU 메모리를 점유하고 있었다. 새로운 학습 프로세스와 GPU 자원을 경합하면서 양쪽 모두 성능이 저하된 것이다. `docker kill`과 `docker rm`으로 좀비 컨테이너를 정리한 후 학습 속도가 원래대로 복구되었다. 이후 학습 시작 전 GPU 점유 상태를 확인하는 절차를 추가했다.

## Ollama GPU 점유

원인 불명의 VRAM 부족이 두 차례 발생했다. `nvidia-smi`를 확인하니 Ollama가 자동 시작되어 GPU VRAM을 약 2GB 점유하고 있었다. 12GB VRAM에서 2GB 감소는 batch size 선택에 직접적 영향을 미쳤다. `taskkill`로 프로세스를 종료하고, Windows 시작프로그램에서 Ollama 자동실행을 비활성화하여 재발을 방지했다. 제한된 GPU 환경에서는 백그라운드 프로세스 하나가 전체 실험 설계를 바꿀 수 있다.

## SageMaker 패키지 충돌

SageMaker SDK v3(3.7.0)를 설치하자 torch가 CPU 버전(2.11.0+cpu)으로 교체되는 문제가 발생했다. SageMaker v3의 의존성 해석기가 torch를 재설치하면서 CUDA 빌드를 무시한 것이다. GPU 학습이 완전히 불가능해졌고, `torch.cuda.is_available()`이 False를 반환했다. SageMaker v2(2.257.1)로 고정 설치하여 기존 torch CUDA 빌드와의 호환성을 유지했다.

## torch 방화벽 차단 복구

관공서 네트워크 환경에서 `download.pytorch.org`가 방화벽에 의해 차단되었다(403 Forbidden). pip install로 torch를 설치할 수 없는 상태였다. conda 캐시 디렉토리(`pkgs/`)에 이전에 다운로드된 `pytorch-2.5.1-cuda12.1` 패키지가 남아 있는 것을 발견하여, 이를 수동으로 복사하여 환경을 재구성했다. 폐쇄망에 가까운 환경에서의 패키지 관리 경험이 되었다.

## PLE Toggle 버그

`use_ple=false` 설정으로 shared_bottom 베이스라인을 실행했는데, 모델 구조를 확인하니 `num_shared_experts=1`, `expert_basket=None`으로 설정되어 있었다. 7개의 이종 전문가가 MLP 1개로 축소된 것이다. shared_bottom(MLP 1개) vs ple_only(expert 7개)는 구조 차이가 너무 커서 PLE의 효과를 측정하는 공정한 비교가 아니었다. `use_ple=false`에서 expert basket은 유지하고 PLE layering만 비활성화하도록 수정하여, 동일 전문가 구성에서 PLE 라우팅의 순수 효과를 비교할 수 있게 했다.

### Batch Size 불일치

ablation 결과에서 일부 시나리오의 학습 시간이 비정상적으로 길었다. 원인을 추적하니, `pipeline.yaml`에는 batch_size=2048이 설정되어 있었지만, `run_ablation_manual.sh`에서 6144로 오버라이드하고 있었다. 일부 시나리오가 6144로 실행되면서 VRAM 23GB까지 사용하여 극심한 속도 저하가 발생했다. 모든 설정을 2048로 통일하여 해결했다.

### VRAM Spillover 분석

batch size에 따른 VRAM 사용 패턴을 정밀 분석했다. batch 6144에서는 전용 메모리 12GB에 공유 GPU 메모리 11GB를 추가 사용하여 총 23GB, 시나리오당 10시간이 소요되었다. batch 4096은 전용 12GB + 공유 3GB = 15GB, batch 2048은 전용 9GB + 공유 0.1GB = 9.1GB로 시나리오당 2시간이었다. 공유 GPU 메모리는 PCIe를 경유하여 전용 VRAM 대비 10-20배 느렸다. 이 분석을 통해 "VRAM 초과 시 느려지는 정도"를 정량적으로 파악하고 최적 batch size를 결정했다.

### Softmax vs Sigmoid Gate 발견

PLE의 val_loss가 Phase 2에서 3.702로 완전히 고정되는 현상을 관찰했다. 동시에 shared_bottom(MLP 1개)의 val_loss=8.08이 ple_only(7 expert)의 val_loss=15.92보다 낮은 역전 현상이 나타났다. CGC(Customized Gate Control)의 softmax gate가 경쟁적 특성을 가져, 이종 전문가 간 수렴을 방해하고 있었다. NeurIPS 2024 논문에서 sigmoid gate가 softmax 대비 이론적 우위를 가진다는 분석을 확인했다. sigmoid 기반 CGC gate를 구현하여 실험을 진행 중이며, 이종 전문가 아키텍처에서 gate 함수 선택이 성능에 결정적 영향을 미칠 수 있다는 새로운 교훈을 얻었다.

---

# 설계 철학의 배경

### "설득의 대상은 항상 사람이다"

금융 추천 시스템의 최종 소비자는 알고리즘이 아니라 사람이다. 고객은 "왜 이 상품인가?"를 묻고, 영업점 직원은 추천 근거를 설명해야 하며, 규제기관은 모델의 의사결정 과정을 검증한다. 확률값 하나만으로는 이 세 그룹 중 누구도 설득할 수 없다. 따라서 모든 설계 결정의 기준은 "이유를 설명할 수 있는가?"였다.

### 2축 분해 프레임워크

아키텍처의 핵심 구조는 2축 분해에 기반한다: **Financial DNA**(이 고객은 누구인가?) × **Data Modality**(데이터의 형태는 무엇인가?). Financial DNA 축은 상품 보유 확률, 다음 상품, 고객 가치, 이탈 위험의 4개 태스크 그룹으로 구성된다. Data Modality 축은 상태/스냅샷/시계열/계층/아이템의 5

가지 피처 유형에 대응하는 이종 전문가로 구성된다. 이 2축의 교차점이 전체 모델의 학습 구조를 정의한다.

## 전문가 붕괴 문제와 이종 전문가의 필요성

동질적 MLP 전문가(예: 모두 같은 구조의 3-layer MLP)를 사용하면 학습 과정에서 전문가 붕괴**(expert collapse)**가 발생한다 — 게이팅 네트워크가 하나의 전문가만 선택하고 나머지는 사실상 사용하지 않는 현상이다. Pinterest와 Kuaishou의 대규모 실험에서도 이 문제가 확인되었다. 구조적으로 서로 다른 이종 전문가(LightGCN, Causal OT, TDA, GMM 등)는 입력 공간과 연산 방식이 다르므로 동일한 표현으로 수렴할 수 없어, 붕괴를 구조적으로 방지한다.

## 피처의 이중 역할: 예측 재료이자 설명 어휘

피처는 예측 성능을 위한 입력일 뿐 아니라, 추천 근거를 설명하는 어휘(vocabulary)이기도 하다. AUC 기여가 미미한 피처라도 영업점에서 "이 고객의 소비 엔트로피가 높아 다양한 상품 경험이 있다"와 같은 설명을 가능하게 한다면, 설명 어휘로서 대체 불가능한 가치를 가진다. 이 때문에 순수한 예측 성능 기준만으로 피처를 제거하지 않았다.

## 경제학에서 데이터 사이언스까지

이 프로젝트의 설계 철학은 PM/리드 아키텍트의 학문적 여정에서 비롯되었다. 경제학을 전공하며 의사결정 과학(Decision Science)을 배웠고, 금융공학을 거쳐 데이터 사이언스에 이르렀다. 이 과정에서 점점 근본적인 의문이 생겼다: **"데이터 기반 방법론에서 과학은 과연 어디에 있는가?"**

경제학은 수백 년의 과학적 방법론 — 가설 수립, 이론적 프레임워크, 반증 가능성 — 을 축적해왔다. 그러나 경제학 자체도 다른 학문에서 도구를 빌려온 역사가 있다: 일반균형이론은 물리학의 열역학 평형에서, 게임이론은 수학의 조합론에서, 계량경제학은 통계학에서 출발했다. 경제학이 과학인 것이 아니라, 학문 간 도구 전이가 과학 발전의 보편적 패턴이다. 실제로 경제학 노벨상 수상자 중 상당수가 물리학 · 수학 출신이다 — Samuelson(열역학→경제균형), Black-Scholes(열전도→옵션가격), Nash(고정점 정리→게임이론), Mandelbrot(프랙탈→금융변동성). 경제학의 가장 강력한 도구들이 다른 과학에서 온 것이다.

그러나 금융공학을 거쳐 데이터 사이언스로 오면서, 과학적 엄밀성은 점점 옅어졌다. 머신러닝 모델은 수학적 구조의 납득이 가능했다 — 선형 회귀의 최소자승법, SVM의 마진 최대화, 트리의 정보 이득 등은 왜 작동하는지 설명할 수 있다.

하지만 딥러닝으로 넘어오면서 상황이 달라졌다. "신경망"이라고 하지만 실제 신경계의 구조를 깊이 연구한 것이 아니라 비유적 명칭에 가깝고, "왜 이 가중치가 이 값인가?"에 대한 답은 "데이터가 그렇게 학습시켰다"뿐이다. 설명할 수 있어야 과학이고, 과학철학이라는 학문까지 생겨서 반증가능성, 패러다임 전환 등을 논했는데, 현재의 딥러닝 접근법은 과학이라기보다 엔지니어링에 가깝다고 느꼈다.

## 구조적 동형사상 — 과학을 다시 가져오기

이 의문에 대한 답이 구조적 동형사상이었다.

이미 인류가 각 학문 분야에서 수백 년에 걸쳐 발견한 과학적 방법론들이 있다. 화학 반응속도론, 역학, 정보이론, 위상수학 — 각각이 주어진 현상에서 어떤 시사점과 인과관계를 끌어낼 수 있는지, 지식의 최전선에서 노력한 결과물이다.

우리가 직면한 문제(금융 고객 행동 이해)의 구조를 제대로 인지했다면, 다른 학문이 이미 풀어놓은 구조적으로 동등한 문제의 해법을 가져올 수 있다. Shannon이 Boltzmann의 열역학 엔트로피를 정보이론에 가져온 것처럼, Black과 Scholes가 열전도 방정식을 옵션 가격에 가져온 것처럼.

이것이 11개 학문 분야의 피처를 도입하고, 각 분야의 수학적 도구에 특화된 이종 전문가를 설계한 근본 동기이다. 단순히 "피처를 많이 만들자"가 아니라, "어떤 과학적 질문을 던질 것인가"가 설계의 출발점이었다.

### 추천 시스템에서 과학적 방법론의 위치

이 프로젝트는 추천 시스템이 단순한 패턴 매칭을 넘어 과학적 이해에 기반할 수 있다는 가능성을 보여주는 시도이다. 모든 추천이 "비슷한 사람들이 이것을 샀다"(상관)에서 끝나는 것이 아니라, "이 고객의 소비 역학이 이러하므로 이 상품이 적합하다"(인과적 설명)로 갈 수 있다.

Pearl의 인과추론, Friedman의 항상소득가설, Boltzmann의 통계역학 — 이들은 각 분야에서 "왜?"에 답한 과학자들이다. 우리의 이종 전문가 아키텍처는 이들의 도구를 금융 추천이라는 맥락에 구조적 동형사상을 통해 가져온 것이며, 이를 통해 추천 시스템에 과학적 설명 가능성을 부여하려는 시도이다.

---

# 핵심 교훈

## "AI가 코드를 쓰지만, 설계 판단은 사람이 한다"

AI 에이전트는 놀라운 속도로 코드를 생성하지만, 아키텍처 결정, 데이터 리키지 판단, 비용 최적화 전략은 사람의 도메인 지식과 경험에 의존한다. AI는 "어떻게(how)"에 강하지만, "왜(why)"와 "무엇을(what)"은 사람이 정의해야 한다.

> *"가장 위험한 순간은 AI가 '그럴듯한 코드'를 만들어낸 직후다.*
> *그때 비판적 검토를 멈추면 기술 부채가 쌓인다."*

## "가드레일 없는 AI 코딩은 기술 부채를 만든다"

CLAUDE.md 없이 AI에게 자유롭게 코딩을 시키면, 하드코딩이 늘어나고 관심사가 뒤섞이며 테스트 불가능한 구조가 만들어진다. 가드레일은 AI의 생산성을 제한하는 것이 아니라, 올바른 방향으로 유도하는 것이다.

### “이종 전문가 철학이 개발 방법론에도 적용된다”

PLE 아키텍처의 핵심 철학인 “이종 전문가(Mixture of Experts)”가 개발 방법론 자체에도 적용되었다. Gemini는 넓은 탐색에, Opus는 깊은 분석에, Cursor는 빠른 환경 구성에, Claude Code는 구현에 각각 전문화되었다. 하나의 AI 도구가 모든 것을 하는 것보다, 각 도구의 강점에 맞는 역할을 배분하는 것이 더 효과적이었다.

# 성과

## 시스템 구축

### 추천 시스템

- 18-task 멀티태스크 학습
- 7-expert PLE 네트워크
- adaTT 태스크 간 적응적 전이
- Uncertainty weighting (Kendall et al.)
- 로짓 전이 3가지 방식

### 인프라 및 실험

- 24개 ablation 시나리오
- SageMaker spot 인스턴스 활용
- Phase 0 (CPU) + Phase 1~2 (GPU) 분리
- Config-driven 파이프라인 아키텍처

## 문서화

프로젝트를 통해 생산된 기술 문서는 총 9편이다. 아키텍처 개요, 파이프라인 가이드, 전문가 상세, 피처 참조, PLE+adaTT 참조, Causal OT 참조, 증류 참조, 시간 참조, 규제 프레임워크가 모두 Typst 기반으로 작성되었다.

## 논문

연구 결과를 정리하여 arXiv에 제출할 논문 2편을 준비했다. 제한된 자원 환경에서의 대규모 멀티태스크 추천 시스템 구축 경험과, AI 에이전트를 활용한 소규모 팀의 개발 방법론을 다룬다. 국내 금융기관 실무자가 arXiv에 논문을 게재하는 최초의 사례가 될 가능성이 있다.

## **Ablation** 분석에서 드러난 전문가 특화

24개 ablation 시나리오 분석 결과, 태스크 유형별로 전문가 특화가 명확히 드러났다. LightGCN은 multiclass 태스크(다음 상품 예측)에서, Causal 전문가는 regression 태스크(고객 가치 추정)에서 가장 큰 기여를 보였다. 이는 이종 전문가 설계의 유효성을 실증적으로 확인한 결과다.

### 온프렘 운영 성과

온프렘 시스템은 규제 준수 관점에서 금감원 AI RMF 24개 항목 중 85% 준수(11개 완전 + 9개 부분)를 달성했으며, 미달 항목은 기술적 구현이 아닌 조직적 의사결정 사항이었다. 12개 규제 준수 모듈(AI 고지, 거부권, 인적 재처리, 공정성 모니터링, 이해충돌 방지, 쏠림 탐지, 프롬프트 인젝션 방어, 안전성 문서, 모델 카드, 감사 추적, 동의 관리, 품질 모니터링)이 구현되었다.

### 평가 지표 체계

태스크 유형별 gold standard 지표를 정립했다: Binary 분류는 AUC, Multiclass 분류는 Macro F1, Regression은 MAE를 기준으로 삼았다. 단일 지표로 모든 태스크를 비교하는 오류를 방지하고, 각 태스크의 특성에 맞는 엄밀한 평가를 수행했다.

## 향후 계획

### 학술 및 업계 발표

- **arXiv** 논문 게재: 2편 (시스템 아키텍처 논문 + AI 에이전트 개발 방법론 논문)
- **Anthropic** 케이스 스터디 제출: Claude Code를 활용한 금융 AI 시스템 구축 사례
- **GARP** 제출: FRM 자격과 AI 리스크 관리를 결합한 관점의 논문

### 규제 및 제도 대응

- 금감원 **AI** 기본법 컴플라이언스 검토 요청: AI 기본법 시행령 및 가이드라인이 수립되는 시점에 맞추어, 본 시스템의 설명 가능성 프레임워크에 대한 검토를 요청할 계획이다.

### 후속 작업

- 온프렘 운영 데이터 결과: 실제 운영 데이터에서의 성능 결과를 논문 보충 자료로 추가
- 공개 **GitHub** 저장소: 조직 정보를 제거한 sanitized 버전의 코드를 공개 저장소로 공개

이 프로젝트는 "자원의 부족"이 아니라 "자원의 재정의"를 통해 완성되었다.
소비자 *GPU 1*대와 *AI* 에이전트들의 조합이 전용 인프라를 대체할 수 있음을 보여준 사례다.